

SMS · SYSTÈME DE MANAGEMENT SCOLAIRE

Guide technique à l'usage de l'ingénieur chargé de la mise en production

Projet : SMS — Système de Management Scolaire

Auteur du projet : Yemeya Luc (LuckyCifer) — Yaoundé,
Cameroun

Stack : Django 5.2 / DRF + React 18 / Vite + MySQL 8

Document généré le : 21 juillet 2026

Sommaire

1. Vue d'ensemble du projet
2. Architecture et stack technique
3. Structure du dépôt
4. Modèle de données
5. Authentification et rôles
6. Isolation multi-établissement (sécurité)
7. Variables d'environnement
8. Procédure de déploiement
9. Checklist de sécurité avant mise en production
10. Tests automatisés
11. Données de démonstration
12. Référence API
13. Problèmes connus et travail restant
14. Dépannage courant

1. Vue d'ensemble du projet

Le SMS (Système de Management Scolaire) est une application web full-stack de gestion d'établissements scolaires, conçue pour le contexte camerounais (établissements primaires, secondaires et supérieurs). Elle couvre la scolarité, les finances, la pédagogie, les examens et la génération de documents officiels (bulletins, cartes étudiantes, diplômes, convocations).

Le système est **multi-établissement** : plusieurs écoles indépendantes partagent la même installation, avec une isolation stricte des données entre elles (voir section 6 — c'est le point de sécurité le plus important à comprendre avant de déployer ce projet).

1.1 Rôles utilisateurs

Sept rôles sont définis, du SUPER_ADMIN (accès total, tous établissements) à l'ÉTUDIANT (lecture seule sur son propre dossier). Le détail complet est en section 5.

1.2 État du projet au moment de ce document

Un audit de sécurité et de préparation au déploiement a été réalisé et les correctifs suivants ont été appliqués :

- Correction d'une faille de contrôle d'accès (IDOR) permettant à un utilisateur authentifié d'accéder aux données d'un autre établissement en falsifiant des en-têtes HTTP.
- Réparation de la suite de tests et de l'historique de migrations Django (30 migrations non rejouables sur une base neuve, squashées en une seule migration propre).
- Durcissement de la configuration Django pour la production (SECRET_KEY, DEBUG, cookies sécurisés, HSTS, CORS).
- Ajout d'un serveur WSGI de production (gunicorn/waitress) et du service des fichiers statiques (whitenoise).
- Extension de la couverture de tests et corrections mineures diverses.

Le détail de ce qui reste à faire est en section 13 — à lire avant la mise en production.

2. Architecture et stack technique

2.1 Backend

Composant	Choix
Framework	Django 5.2.12
API	Django REST Framework 3.17
Authentification	djangorestframework-simplejwt (JWT)
Base de données	MySQL 8, connecteur <code>mysqlclient</code>
Python	3.11.x — ne pas utiliser 3.14 , incompatible avec les dépendances actuelles
Fichiers statiques	whitenoise (servis directement par Django, pas de nginx séparé nécessaire)
Serveur WSGI production	gunicorn (Linux) / waitress (Windows) — sélection automatique selon l'OS via <code>requirements.txt</code>

2.2 Frontend

Composant	Choix
Framework	React 18 + Vite 7
UI	AdminLTE 3.2 (thème custom, palette ACERFI)
HTTP	Axios avec intercepteurs JWT
Routage	react-router-dom 6, pages chargées à la demande (<code>React.lazy</code>)
Graphiques	chart.js / react-chartjs-2

Palette de couleurs : `#1F3864` (bleu foncé, primaire) · `#2E74B5` (bleu moyen) · `#D6E4F0` (bleu clair, accents)

2.3 Base de données

Nom	<code>sms</code>
Encodage	utf8mb4 / <code>utf8mb4_unicode_ci</code> (collation uniforme requise — voir encadré)
Host	<code>127.0.0.1</code> — ne pas utiliser <code>localhost</code> (résolution différente avec mysqlclient sur certains systèmes)
Port	3306

Point technique important sur la collation MySQL

MySQL 8 utilise par défaut la collation `utf8mb4_0900_ai_ci`, incompatible pour les clés étrangères avec `utf8mb4_unicode_ci`. Le projet a été corrigé pour que toutes les tables utilisent uniformément `utf8mb4_unicode_ci` (voir `DATABASES.OPTIONS` et `DATABASES.TEST` dans `settings.py`). Si vous créez la base de données manuellement, assurez-vous de la créer avec cette collation : `CREATE DATABASE sms CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;`

3. Structure du dépôt

```
Projet SMS/
├── backend/
│   ├── api/
│   │   ├── models.py           # ~40 modèles (tables)
│   │   ├── serializers.py      # Serializers DRF
│   │   ├── views.py            # ViewSets + logique métier (~2500 lignes)
│   │   ├── urls.py             # Routes API (DefaultRouter)
│   │   ├── admin.py            # Interface admin Django
│   │   ├── mixins.py           # EtablissementFilterMixin – isolation multi-étap (§6)
│   │   ├── pagination.py       # Pagination DRF (page_size=20)
│   │   ├── permissions.py      # (n'existe pas – permissions inline dans views.py)
│   │   ├── utils.py            # get_client_ip, log_action (audit)
│   │   ├── imports.py          # Import CSV en masse
│   │   ├── image_utils.py      # Traitement des photos/logos
│   │   └── management/commands/
│   │       ├── seed_demo.py    # Jeu de données de démonstration (§11)
│   │       └── migrations/
│   │           └── 0001_initial.py # Migration unique (squashée, voir §13)
│   │       └── tests/          # Suite de tests (§10)
│   ├── sms_backend/            # Config Django (settings, urls, wsgi)
│   ├── requirements.txt
│   ├── .env.example            # Modèle de configuration – à copier en .env
│   └── manage.py
├── frontend/
│   ├── src/
│   │   ├── components/        # Composants réutilisables
│   │   ├── pages/             # ~30 pages, une par module métier
│   │   ├── layouts/           # Layout AdminLTE (Navbar, Sidebar)
│   │   ├── services/          # api.js (axios), endpoints.js
│   │   ├── context/           # AppContext, NotificationContext
│   │   ├── hooks/             # useApi, useEtablissement...
│   │   └── utils/              # roles.js (permissions), i18n
│   ├── vite.config.js
│   └── package.json
└── CLAUDE.md                   # Contexte projet (pour outils IA)
```

4. Modèle de données

Le schéma compte une quarantaine de tables, organisées en six familles fonctionnelles. La clé primaire des entités métier est généralement un code métier (`code_etab` , `mle_etudiant` ...), tandis que les tables transactionnelles utilisent un auto-incrément.

4.1 Référentiels

Modèle	Table	Rôle
TypeEtab	type_etab	Types d'établissement
Batiment / Salle	batiment / salle	Locaux
Jour / Langue / Module	—	Référentiels pédagogiques
Pension / Mention	—	Types de pension, mentions (TB, B, AB...)
TypeEvaluation / Rapport	—	Types d'évaluation, de rapport

4.2 Structure académique

Modèle	Clé primaire	Rôle
Annee	code_annee	Années académiques
Etablissement	code_etab	Écoles — racine de l'isolation multi-tenant
Faculte / Departement / Specialite	codes propres	Filières
Cycle / Niveau / Classe	codes propres	Organisation pédagogique

4.3 Acteurs

Modèle	Clé primaire	Rôle
Etudiant	mle_etudiant	Élèves/étudiants
Enseignant	mle_ens	Enseignants
Personnel	mle_personnel	Personnel administratif (directeur, censeur, économiste...)
Utilisateur	login	Comptes système (distinct de <code>django.contrib.auth.User</code> , voir §5)
Tuteur	auto	Parents/tuteurs

4.4 Scolarité et finances

Modèle	Rôle
Inscription / FraisInscription	Inscriptions et frais associés
Tranche / Frais	Barème des frais scolaires
Paielement / Moratoire	Paielements et échéanciers
Facture / FactureDetail	Facturation
RapportFinancier	Rapports financiers

4.5 Pédagogie

Modèle	Rôle
Matiere / Qualification	Matières et qualifications enseignants
Cours / UniteEnseignement / Periode	Attribution des cours, semestres
Evaluation / FicheNotes(Detail)	Notes et fiches de notes
Planning / Seance / Absence	Emploi du temps (HEBDO/INTENSIF), présence

4.6 Examens et documents

Modèle	Rôle
Examen / Convocation / Decision	Organisation des examens et délibérations
Diplome / CarteEtudiant / LettreAdmission / BadgeAcces	Documents officiels générés
DocumentGenere / AuditLog	Traçabilité des documents et des actions

Champ commun d'isolation

La plupart des tables métier possèdent un champ **etablissement** (FK vers **Etablissement**), ajouté lors de la migration vers le multi-établissement. C'est ce champ que le mixin d'isolation décrit en section 6 utilise pour filtrer automatiquement les données.

5. Authentification et rôles

5.1 Deux notions d'utilisateur à ne pas confondre

Le projet utilise **deux modèles utilisateur en parallèle**, liés par le *login* :

- `django.contrib.auth.User` (standard Django) — porte l'authentification technique et les tokens JWT.
- `api.Utilisateur` (métier, table `utilisateur`) — porte le rôle, l'établissement assigné et le mot de passe applicatif (hashé). C'est ce modèle qui pilote toute l'autorisation métier.

Le lien se fait par `Utilisateur.objects.get(login=request.user.username)`, utilisé systématiquement dans les vues et permissions personnalisées.

5.2 Flux d'authentification

```
POST /api/auth/sms-login/ { "login": "...", "password": "..." }  
  → vérifie Utilisateur.passwd (hashé avec make_password/check_password)  
  → crée ou récupère le django.contrib.auth.User correspondant  
  → retourne { access, refresh } (JWT, via SimpleJWT)
```

Requêtes suivantes : Authorization: Bearer <access>

POST /api/auth/refresh/ → renouvelle le token access

POST /api/auth/sms-logout/ → blackliste le refresh token

Paramètre JWT	Valeur
ACCESS_TOKEN_LIFETIME	24 heures
REFRESH_TOKEN_LIFETIME	30 jours
ROTATE_REFRESH_TOKENS / BLACKLIST_AFTER_ROTATION	Activés

5.3 Rôles et permissions (frontend — `utils/roles.js`)

Rôle	Pages accessibles	Créer	Modifier	Supprimer	Change d'étab.
SUPER_ADMIN	Toutes (*)	✓	✓	✓	✓
ADMIN	Toutes (*)	✓	✓	✓	—
SCOLARITE	students, classes, cours, inscription, evaluation, grades, absences, bulletins, importCsv, personnel	✓	✓	✓	—
ENSEIGNANT	classes, evaluation, grades, cours, absences	✓	✓	—	—
ETUDIANT	grades, cours, monDossier	—	—	—	—
COMPTABLE	payments, inscription	✓	✓	—	—
DIRECTION	reports, statistics, absences, bulletins, personnel	—	—	—	—

Le contrôle frontend n'est qu'un confort d'UX

`roles.js` masque les pages non pertinentes dans l'interface, mais

n'est pas une mesure de sécurité

— toute autorisation réelle doit être (et est, pour les points vérifiés) appliquée côté serveur (DRF

`permission_classes`).

5.4 Règle particulière SUPER_ADMIN

Le SUPER_ADMIN n'est rattaché à aucun établissement fixe (`etablissement` et `type_etab` nullables) et peut naviguer entre tous les établissements via un sélecteur d'interface (en-têtes `X-Etablissement-Id` / `X-Type-Etab` , voir §6). Tous les autres rôles sont rattachés à un seul établissement ; pour en gérer plusieurs, un utilisateur doit disposer d'un compte par établissement.

6. Isolation multi-établissement (sécurité)

À lire impérativement avant toute modification de la logique d'accès aux données

Une faille de contrôle d'accès (IDOR) a été identifiée et corrigée dans ce mécanisme durant l'audit. Comprendre son fonctionnement actuel est essentiel pour ne pas la réintroduire.

6.1 Principe

Toutes les données métier (élèves, notes, paiements...) sont rattachées à un `Etablissement` via un champ `etablissement`. Le frontend indique le contexte courant via deux en-têtes HTTP :

- `X-Etablissement-Id` — l'établissement actif
- `X-Type-Etab` — le type d'établissement actif (réservé au SUPER_ADMIN)

6.2 Mécanisme corrigé — `api/mixins.py`

Avant correctif, ces en-têtes étaient **honorés tels quels**, quel que soit l'utilisateur — un compte ENSEIGNANT ou ÉTUDIANT authentifié pouvait donc lire ou modifier les données d'une autre école en falsifiant simplement `X-Etablissement-Id` dans ses requêtes.

La correction introduit trois fonctions faisant autorité dans `api/mixins.py` :

```
get_utilisateur_profile(request)  # résout le profil Utilisateur du compte connecté
is_super_admin(request)          # True uniquement si profil.role == 'SUPER_ADMIN'
trusted_etablissement_header(request)
    # SUPER_ADMIN → valeur réelle de l'en-tête client
    # tout autre rôle → TOUJOURS profil.etablissement_id, header ignoré
trusted_type_etab_header(request)
    # SUPER_ADMIN → valeur réelle de l'en-tête client
    # tout autre rôle → toujours '' (ignoré)
```

`EtablissementFilterMixin` et **tous** les points d'accès qui lisaient auparavant ces en-têtes directement (`UtilisateurViewSet`, `PlanningViewSet`, `DashboardStatsView`, l'import CSV, l'export Excel, `EtablissementViewSet.current` ...) passent désormais par ces fonctions.

6.3 Refus par défaut

Autre changement important : si l'établissement d'un utilisateur non-SUPER_ADMIN ne peut pas être déterminé, le comportement est un **refus par défaut** (queryset vide) plutôt qu'un repli silencieux sur « le premier établissement actif » — l'ancien comportement qui constituait lui aussi une fuite potentielle.

6.4 Règle à respecter pour tout nouveau code

Ne jamais lire `request.META['HTTP_X_ETABLISSEMENT_ID']` ou `HTTP_X_TYPE_ETAB` directement dans une nouvelle vue. Toujours passer par `trusted_etablissement_header(request)` / `trusted_type_etab_header(request)` importées depuis `api.mixins`. Un test de non-régression existe dans `api/tests/test_isolation.py` pour détecter toute réintroduction de la faille.

7. Variables d'environnement

7.1 Backend — backend/.env

Copier `backend/.env.example` vers `backend/.env` et renseigner :

Variable	Défaut si absente	Notes
<code>SECRET_KEY</code>	<i>aucun en production</i>	Obligatoire si <code>DEBUG=False</code> — l'application refuse de démarrer sans elle. Générer avec la commande ci-dessous.
<code>DEBUG</code>	<code>False</code>	Ne jamais mettre <code>True</code> en production (expose les stack traces).
<code>DB_NAME</code>	<code>sms</code>	
<code>DB_USER</code>	<code>root</code>	
<code>DB_PASSWORD</code>	vide	
<code>DB_HOST</code>	<code>127.0.0.1</code>	Pas <code>localhost</code>
<code>DB_PORT</code>	<code>3306</code>	
<code>ALLOWED_HOSTS</code>	<code>localhost,127.0.0.1</code>	Domaines séparés par des virgules. Ignoré si <code>DEBUG=True</code> (autorise tout).
<code>CORS_ALLOWED_ORIGINS</code>	<code>localhost:5173</code>	À définir en production avec le(s) domaine(s) du frontend, séparés par des virgules.
<code>SECURE_SSL_REDIRECT</code>	<code>True</code>	Actif seulement si <code>DEBUG=False</code> .
<code>SECURE_HSTS_SECONDS</code>	<code>31536000</code> (1 an)	Actif seulement si <code>DEBUG=False</code> .
<code>BEHIND_HTTPS_PROXY</code>	<code>False</code>	À mettre <code>True</code> uniquement si un reverse-proxy (nginx...) termine le TLS et transmet <code>X-Forwarded-Proto</code> .
<code>EMAIL_*</code>	backend console (dev)	Configurer host/user/password SMTP en production.

```
# Générer un SECRET_KEY de production :  
python -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"
```

7.2 Frontend — frontend/.env

Variable	Défaut	Notes
VITE_API_URL	http://localhost:8000	URL du backend — à définir vers le domaine de production avant <code>npm run build</code> (la valeur est injectée au build, pas au runtime).

8. Procédure de déploiement

8.1 Prérequis

- Python 3.11.x (pas 3.14)
- Node.js 18+ (pour builder le frontend)
- MySQL 8 accessible
- Un serveur Linux (recommandé) ou Windows Server

8.2 Base de données

```
CREATE DATABASE sms CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Aucun import SQL manuel n'est nécessaire : les migrations Django créent tout le schéma.

8.3 Backend Django

```
cd backend
python -m venv venv
source venv/bin/activate          # Linux/Mac
# venv\Scripts\activate           # Windows

pip install -r requirements.txt    # installe gunicorn OU waitress selon l'OS automatiquement

cp .env.example .env
# → éditer .env : SECRET_KEY, DEBUG=False, DB_*, ALLOWED_HOSTS, CORS_ALLOWED_ORIGINS

python manage.py migrate
python manage.py collectstatic --noinput
python manage.py createsuperuser    # compte admin Django (interface /admin/)
# Optionnel : python manage.py seed_demo → jeu de données de démo (voir §11)

# Démarrage production :
gunicorn sms_backend.wsgi:application --bind 0.0.0.0:8000 --workers 3    # Linux
# waitress-serve --port=8000 sms_backend.wsgi:application                # Windows
```

Ne jamais utiliser `python manage.py runserver` en production

— c'est un serveur de développement, non sécurisé et non conçu pour tenir la charge.

8.4 Frontend React

```
cd frontend
npm install
echo "VITE_API_URL=https://api.votre-domaine.cm" > .env
npm run build
# → génère frontend/dist/, à servir en fichiers statiques (nginx, Vercel, Netlify, etc.)
```

8.5 Reverse-proxy (recommandé)

Une configuration nginx typique : servir `frontend/dist/` directement, et proxyfier `/api/` et `/admin/` vers gunicorn (port 8000). Si nginx termine le TLS, positionner `BEHIND_HTTPS_PROXY=True` côté backend pour que Django reconnaisse correctement les requêtes HTTPS transmises en HTTP en interne.

8.6 Fichiers médias

`MEDIA_ROOT` (`backend/media/`) contient les photos d'élèves, logos d'établissements, etc. À sauvegarder / monter sur un volume persistant si le serveur est éphémère (conteneur, PaaS).

9. Checklist de sécurité avant mise en production

- ☐ `DEBUG=False` dans le `.env` de production
- ☐ `SECRET_KEY` généré aléatoirement (pas la valeur par défaut du dépôt)
- ☐ `ALLOWED_HOSTS` limité au(x) domaine(s) réel(s)
- ☐ `CORS_ALLOWED_ORIGINS` limité au(x) domaine(s) du frontend
- ☐ HTTPS actif (certificat valide) — `SECURE_SSL_REDIRECT=True`
- ☐ Si derrière un reverse-proxy TLS : `BEHIND_HTTPS_PROXY=True`
- ☐ Mot de passe MySQL non vide en production (`DB_PASSWORD`)
- ☐ `python manage.py check --deploy` exécuté sans avertissement
- ☐ `python manage.py test api` passe (voir §10)
- ☐ Sauvegardes automatiques de la base MySQL configurées
- ☐ Fichier `.env` exclu du contrôle de version et des logs de build
- ☐ Vérifier qu'aucun fichier `.env` réel n'a été commité par erreur (seul `.env.example` doit être versionné)

Vérification rapide

— cette commande doit afficher « System check identified no issues » avec les variables de production positionnées :

```
DEBUG=False SECRET_KEY=<clé générée> python manage.py check --deploy
```

10. Tests automatisés

```
cd backend
python manage.py test api --noinput
```

La suite compte 24 tests couvrant :

Fichier	Contenu
<code>test_models.py</code>	Modèles de base (Étudiant, Département, Spécialité)
<code>test_views.py</code>	Authentification (login SMS), accès non authentifié
<code>test_finance.py</code>	Année, Frais, Paiement, Facture (contrainte d'unicité)
<code>test_permissions.py</code>	Permissions par rôle (création Utilisateur, modification établissement)
<code>test_isolation.py</code>	Non-régression de l'isolation multi-établissement — vérifie qu'un en-tête falsifié ne donne pas accès aux données d'une autre école

Ne pas supprimer `test_isolation.py`

— c'est le filet de sécurité qui détecterait une régression sur la faille corrigée en section 6.

La base de test est créée et détruite automatiquement à chaque exécution (base `test_sms`) ; elle nécessite les mêmes accès MySQL que la base de développement.

11. Données de démonstration

```
python manage.py seed_demo
```

Commande idempotente (relançable sans risque) qui crée 3 établissements complets (primaire, secondaire, supérieur) avec structures académiques, élèves et comptes utilisateurs de test pour chaque rôle.

Mot de passe des comptes de démo :

Demo2026! pour tous les comptes créés par cette commande.

À ne jamais utiliser en production

— ces comptes sont prévus pour des démonstrations/soutenances, pas pour un environnement réel.

12. Référence API

12.1 Authentification

Endpoint	Méthode	Description
<code>/api/auth/sms-login/</code>	POST	Connexion (login + password) → JWT
<code>/api/auth/refresh/</code>	POST	Renouvellement du token access
<code>/api/auth/verify/</code>	POST	Vérification d'un token
<code>/api/auth/sms-logout/</code>	POST	Déconnexion (blacklist du refresh token)

12.2 Ressources principales (REST, DefaultRouter)

Toutes suivent le pattern standard DRF (`GET/POST /api/<ressource>/` , `GET/PUT/PATCH/DELETE /api/<ressource>/<id>/`), paginées (20 par page), filtrables et exportables en CSV/XLSX pour la plupart (paramètre `?format=csv`).

Domaine	Endpoints (préfixe <code>/api/</code>)
Structure	etablissements, facultes, departements, specialites, cycles, niveaux, classes
Acteurs	etudiants, enseignants, utilisateurs, tuteurs, personnel
Finances	tranches, frais, inscriptions, paiements, moratoires, factures, rapports-financiers
Pédagogie	matieres, qualifications, cours, periodes, evaluations, fiches-notes
Planning	planning, rapports-cours, seances, absences
Examens	examens, convocations, rapports-stat, decisions, diplomes
Divers	type-etab, batiments, salles, jours, langues, modules, pensions, mentions

Endpoints spéciaux notables : `GET/PUT/PATCH /api/etablissements/current/` (établissement de l'utilisateur connecté — écriture réservée ADMIN/SUPER_ADMIN), `GET /api/dashboard/stats/` (KPIs consolidés), `POST /api/import-csv/<type>/` (import en masse).

12.3 En-têtes de contexte

En-tête	Usage
<code>Authorization: Bearer <token></code>	Obligatoire sur tous les endpoints sauf login
<code>X-Etablissement-Id</code>	Établissement actif — voir §6, honoré uniquement pour SUPER_ADMIN
<code>X-Type-Etab</code>	Type d'établissement actif — réservé SUPER_ADMIN

13. Problèmes connus et travail restant

13.1 À traiter avant/pendant le déploiement

Fichiers non commités

Au moment de la rédaction de ce document, le dépôt contient un nombre important de fichiers modifiés ou nouveaux non encore commités (corrections de sécurité, migrations squashées, nouvelles pages frontend, commandes de seed...).

Vérifier `git status` et s'assurer qu'un commit propre existe avant de baser un déploiement sur une branche donnée

— ne pas déployer un état de travail non versionné.

13.2 Dette technique mineure (non bloquante)

- **Couverture de tests partielle** — les modules Cours/Planning/Évaluation (plus lourds en dépendances de fixtures) ne sont pas encore couverts par des tests automatisés.
- **`api/permissions.py` n'existe pas** — les classes de permission personnalisées (`IsAdminOrSuperAdmin`, etc.) sont définies directement en haut de `views.py` plutôt que dans un module dédié. Fonctionnel, mais à extraire si le fichier continue de grossir.
- **Historique de migrations compact** — l'historique détaillé des évolutions de schéma (30 migrations) a été remplacé par une seule migration reflétant l'état final des modèles (voir §8). C'est correct et testé pour tout nouveau déploiement, mais signifie qu'il n'existe plus de trace migration-par-migration de l'évolution historique du schéma.

13.3 Historique des correctifs de sécurité appliqués

Problème	Statut
IDOR — accès inter-établissements via en-têtes falsifiés	✅ Corrigé (§6) + test de non-régression
Migrations non rejouables sur base neuve	✅ Corrigé — squashées en <code>0001_initial.py</code>
Conflit <code>api/tests.py</code> / <code>api/tests/</code>	✅ Corrigé
DEBUG=True par défaut, SECRET_KEY codé en dur	✅ Corrigé (§7, §9)
Pas de serveur WSGI ni de service des statiques en production	✅ Corrigé (§8.3)
<code>EtablissementViewSet</code> sans contrôle de rôle en écriture	✅ Corrigé
Bundle frontend > 500 Ko / pas de code-splitting	✅ Corrigé (routes en <code>React.lazy</code>)

14. Dépannage courant

Problème	Cause probable	Solution
<code>django.db.utils.OperationalError</code> au démarrage	MySQL non démarré ou inaccessible	Vérifier le service MySQL et les identifiants <code>DB_*</code>
Token JWT expiré (401 partout)	Session locale expirée	Vider le localStorage frontend, se reconnecter
<code>mysqlclient</code> / <code>MySQLdb</code> introuvable	Mauvais environnement Python actif	Activer le venv avec Python 3.11 et réinstaller <code>requirements.txt</code>
Erreur de clé étrangère MySQL 1005/3780 lors d'un <code>migrate</code>	Collation incohérente entre tables	Vérifier que la base a bien été créée avec <code>utf8mb4_unicode_ci</code> (§2.3)
CORS bloqué dans le navigateur	<code>CORS_ALLOWED_ORIGINS</code> ne contient pas le domaine du frontend	Ajouter le domaine exact (avec protocole) dans le <code>.env</code> backend
<code>ImproperlyConfigured: SECRET_KEY doit être défini</code>	DEBUG=False sans SECRET_KEY dans l'environnement	Définir <code>SECRET_KEY</code> dans <code>.env</code> (voir §7.1)
Fichiers statiques non servis (404 sur <code>/static/...</code>)	<code>collectstatic</code> non exécuté	<code>python manage.py collectstatic --noinput</code>